

dynamicVSstatic.pdf



abcd__1234



Paralelismo



3º Grado en Ingeniería Informática



Facultad de Informática de Barcelona (Fib)
Universidad Politécnica de Catalunya



[Accede al documento original](#)

Poliamor 🖐️

Ahora que tenemos tu atención: en ING puedes abrir **hasta 3 cuentas gratuitas***. Para ti o para compartir con quien quieras 😊 ¡No juzgamos!

Saber más

1/6

Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANKING se encuentra adherido al Sistema de Garantía de Depósitos holandés con una garantía de hasta 100.000 euros por depositante. Consulta más información en [ing.es](#)

*TIN 0 % y TAE 0 %.



Si estás en tu spending era...

mejor tener una app que te diga en qué
tiendas se ha quedado registrada tu tarjeta.
¡Como la app de ING!

Saber más



Qualificació 3,00 sobre 3,00 (100%)

Pregunta 1

Completa

Puntuació 1,00
sobre 1,00

Marca la
pregunta

Assume that the 128 elements of vectors a and b are distributed across the 4 nodes (M_0 - M_3) of a NUMA multiprocessor system as shown in the following figure and explained below:

M_0	M_1	M_2	M_3
0..31	32..63	64..95	96..127

That the first block of 32 consecutive elements is stored in M_0 , the second block of 32 consecutive elements is stored in M_1 , the third block of 32 consecutive elements is stored in M_2 , and the last block of 32 consecutive elements is stored in M_3 .

Each NUMA node consists of a single processor with its own local cache hierarchy and a portion of the physically distributed but logically shared main memory. Caches are kept coherent across (NUMA) nodes by using a directory-based coherence protocol. Each cache and memory line has a number of bits sufficient to store 4 consecutive elements of these vectors. The first element of vectors a and b is aligned to the beginning of a cache line.

Given the following OpenMP parallel region (thread with OpenMP identifier i executes in node M_i):

```
#pragma omp parallel num_threads(4)
#pragma omp single
for (iter=0; i<99; iter++) {
    #pragma omp taskloop num_tasks(4)
    for (int i=0; i<128; i++)
        b[i] = foo1(a[i]);
    #pragma omp taskloop num_tasks(4)
    for (int i=0; i<128; i++)
        a[i] = foo2(b[i]);
}
```

Which of the following statements is correct?

Trieu-ne una:

- ☐ The execution of this parallel region does not cause coherence traffic; in other words, since each node has a portion of the vectors and the number of tasks generated in each taskloop equals the number of nodes, there will be no coherence commands

```
for (iter=0; i<99; iter++) {  
    #pragma omp taskloop num_tasks(4)  
    for (int i=0; i<128; i++)  
        b[i] = foo1(a[i]);  
    #pragma omp taskloop num_tasks(4)  
    for (int i=0; i<128; i++)  
        a[i] = foo2(b[i]);  
}
```

Which of the following statements is correct?

Tríeune una:

- ☐ The execution of this parallel region does not cause coherence traffic; in other words, since each node has a portion of the vectors and the number of tasks generated in each taskloop equals the number of nodes, there will be no coherence commands interchanged between them.
- ☐ The execution of this parallel region only causes coherence traffic during the execution of the first iteration (iter=0) of the outer loop; in other words, once the elements are brought into cache there will be no coherence commands interchanged between nodes.
- ☒ The execution of this parallel region may cause coherence traffic during the execution of all iterations of the outer loop; in other words, it is highly probable that the processor in a node accesses (reads and/or writes) data cached in the cache of other nodes.



Pregunta 2

Completa

Puntuació 1,00
sobre 1,00

 Marca la
pregunta

For the same architecture, if the parallel region is changed as follows (assuming thread with OpenMP identifier i executes in node M_i):

```
#pragma omp parallel num_threads(4)
{
    int myid = omp_get_thread_num();
    int BS = 128 / omp_get_num_threads();

    for (iter=0; i<99; iter++) {
        for (int i=myid*BS; i<(myid+1)*BS; i++)
            b[i] = foo1(a[i]);
        for (int i=myid*BS; i<(myid+1)*BS; i++)
            a[i] = foo2(b[i]);
    }
}
```

Which of the following statements is correct?

Trieu-ne una:

- ☒ The execution of this parallel region does not cause coherence traffic because each node has a portion of the vectors and the static assignment will always assign the same iterations to each thread. Thus, each thread will only access the portion of the vectors stored within the memory of a node. Consequently, there will be no coherence commands interchanged between nodes
- ☐ The execution of this parallel region only causes coherence traffic during the execution of the first iteration ($\text{iter}=0$) of the outer loop; in other words, once the elements are brought into cache there will be no coherence commands interchanged between nodes.
- ☐ The execution of this parallel region may cause coherence traffic during the execution of all iterations of the outer loop; in other words, it is highly probable that the processor in a node accesses (reads and/or writes) data cached in the cache of other nodes.



WUOLAH

Consigue tu Beca



INSCRÍBETE
GRATIS

FIEP

FERIA
INTERNACIONAL
DE ESTUDIOS
DE POSTGRADO

Feria de
Postgrado
FIEP-
Barcelona

Hotel NH
Collection
Constanza

jueves, 26 de
febrero de 2026,
16:00 - 19:00

Pregunta 3

Completa

Puntuació 1,00
sobre 1,00

 Marca la
pregunta

Assume now that the 128 elements of vectors a and b are allocated only in the node associated to M_0 . This has happened because 1) the programmer simply forgot to parallelize the initialization loop:

```
for (int i=0; i<128; i++) {  
    a[i] = random();  
    b[i] = random();  
}
```

and 2) because of the usual "first touch policy" that is applied in NUMA systems. As in the previous two questions, assume thread with OpenMP identifier i executes in node M_i . Compared to the previous two questions, in which vector elements were distributed among all the nodes, which of the following statements is true:

Trieu-ne una o més:

- ☐ The two previous versions of the parallel region (the one with taskloop and the other with static schedule) will not cause now any coherence traffic since vectors are stored in a single node (M_0) of the system.
- ☒ The version based on taskloop (in Question 1) will now behave even worse since all coherence commands should be served by the same directory structure in node M_0 .
- ☒ The code in Question 2, which does a static schedule of iterations to each thread, will have a similar behaviour after the execution of the first iteration ($iter=0$) of the outer loop. However, during the first iteration of the outer loop the behaviour will be worse now since all coherence commands should be served by the same directory structure in node M_0 .

WUOLAH